

Министерство образования Республики Беларусь
Учреждение образования «Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5

По дисциплине «Модели решения задач в ИС»

Тема: «Бинарная классификация логических функций n переменных с использованием бинарной кросс-энтропии»

Выполнил:

Студент 3 курса

Группы ИИ-26 (2)

Яшков П.А.

Проверила:

Андренко К.В.

Цель работы: Реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

Вариант: тот же, что в Лабораторной №4 (логическая функция и значение n).

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для своей логической функции (2^n примеров).
2. Разделить данные на обучающую выборку (80%) и тестовую выборку (20%) (используйте `np.random.seed(42)` для воспроизводимости).
3. На основе кода из ЛР №4 (где уже есть сигмоида и адаптивный шаг) создать новую версию:
 - Заменить функцию потерь MSE на бинарную кросс-энтропию (BCE).
 - Реализовать правильный градиент для BCE + сигмоида:
4. Реализовать четыре конфигурации обучения (все в онлайн-режиме — один пример за обновление, как в предыдущих лабах):
 - А. MSE + фиксированный шаг (результаты можно взять из ЛР №4)
 - Б. MSE + адаптивный шаг (из ЛР №4)
 - В. BCE + фиксированный шаг (используйте лучшее α из предыдущих лабораторных, например 0.1 или 0.5)
 - Г. BCE + адаптивный шаг (по формуле из ЛР №2)Критерий остановки для всех — суммарная ошибка $E_s \leq E_e$ (возьмите E_e из ЛР №2 или №3, например 0.05 или 0.01).
5. Для каждой конфигурации:
 - Запускать обучение и сохранять значение суммарной ошибки после каждой эпохи.
 - Зафиксировать количество эпох до достижения критерия остановки.
 - После обучения посчитать ассигасу на обучающей и тестовой выборках.
 - Проверить, сколько процентов из всей таблицы истинности (2^n примеров) сеть предсказывает правильно.
6. Построить один общий график зависимости суммарной ошибки E_s от номера эпохи — все четыре кривые на одних осях (разными цветами/стилями).
7. Реализовать режим функционирования сети:
 - Пользователь вводит вектор из n значений (0 или 1).
 - Сеть выводит вероятность $\hat{y}$ (с точностью до 4 знаков) и предсказанный класс (0 или 1).
 - Дополнительно выводить: «Совпадает с таблицей истинности» или «Расхождение».
8. Написать вывод (обязательно ответить на следующие пункты):
 - Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?
 - Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?
 - Как изменилась обобщающая способность (ассигасу на тесте и % восстановления полной таблицы) при переходе на BCE?
 - Как влияет адаптивный шаг на обучение с BCE по сравнению с

фиксированным шагом? • Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

Допускается: NumPy, Matplotlib

Запрещено: любые ML-фреймворки (scikit-learn, PyTorch, TensorFlow и т.д.)

Вариант 25

№ of variant	n	Task
25	11	OR

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
n = 11
Ee = 0.01      # критерий останковки
max_epochs = 500 # максимум эпох
lr_fixed = 0.1  # фиксированный шаг обучения

def generate_truth_table(n):
    X = np.array([list(map(int, format(i, f'0{n}b')))) for i in range(2 ** n)])
    y = np.any(X == 1, axis=1).astype(int).reshape(-1, 1) # OR
    return X, y

def train_test_split_manual(X, y, train_ratio=0.8):
    indices = np.arange(len(X))
    np.random.shuffle(indices)

    split_idx = int(len(X) * train_ratio)
    train_idx = indices[:split_idx]
    test_idx = indices[split_idx:]

    X_train = X[train_idx]
    y_train = y[train_idx]
    X_test = X[test_idx]
    y_test = y[test_idx]

    return X_train, X_test, y_train, y_test

def sigmoid(z):
    z = np.clip(z, -500, 500) # защита от переполнения
    return 1.0 / (1.0 + np.exp(-z))
```

```

def mse_loss(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def bce_loss(y_true, y_pred):
    eps = 1e-12
    y_pred = np.clip(y_pred, eps, 1 - eps)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

def update_learning_rate(current_lr, prev_error, curr_error):
    if curr_error < prev_error:
        return current_lr * 1.05
    else:
        return current_lr * 0.7

def train_model(X, y, loss_type="mse", adaptive=False, lr=0.1, max_epochs=500, Ee=0.01):
    # Инициализация весов
    w = np.random.randn(n, 1) * 0.1
    b = np.random.randn(1)[0] * 0.1

    errors = []
    lr_current = lr

    for epoch in range(max_epochs):
        total_error = 0.0

        # Онлайн-режим: 1 пример -> 1 обновление
        for i in range(len(X)):
            xi = X[i].reshape(-1, 1)    # (n, 1)
            yi = y[i].reshape(1, 1)    # (1, 1)

            z = w.T @ xi + b
            y_pred = sigmoid(z)

            # Ошибка
            if loss_type == "mse":
                sample_error = mse_loss(yi, y_pred)
                # dE/dz для MSE + sigmoid
                delta = (y_pred - yi) * y_pred * (1 - y_pred)
            elif loss_type == "bce":
                sample_error = bce_loss(yi, y_pred)
                # dE/dz для BCE + sigmoid
                delta = (y_pred - yi)
            else:
                raise ValueError("Неизвестный тип функции потерь")

            # Обновление параметров
            w -= lr_current * xi * delta

```

```

        b -= lr_current * delta.item()

        total_error += sample_error

        # Средняя суммарная ошибка за эпоху
        total_error /= len(X)
        errors.append(total_error)

        # Адаптивный шаг
        if adaptive and epoch > 0:
            lr_current = update_learning_rate(lr_current, errors[-2], errors[-1])

        # Критерий остановки
        if total_error <= Ee:
            break

    return w, b, errors, epoch + 1

def predict(X, w, b):
    probs = sigmoid(X @ w + b)
    classes = (probs >= 0.5).astype(int)
    return probs, classes

def accuracy_score(y_true, y_pred):
    return np.mean(y_true == y_pred)

def evaluate_model(X, y, w, b):
    probs, classes = predict(X, w, b)
    acc = accuracy_score(y, classes)
    return acc

X, y = generate_truth_table(n)
X_train, X_test, y_train, y_test = train_test_split_manual(X, y, train_ratio=0.8)

configs = [
    ("MSE + fixed", "mse", False),
    ("MSE + adaptive", "mse", True),
    ("BCE + fixed", "bce", False),
    ("BCE + adaptive", "bce", True),
]

results = {}

for name, loss_type, adaptive in configs:
    w, b, errors, epochs = train_model(
        X_train,

```

```

        y_train,
        loss_type=loss_type,
        adaptive=adaptive,
        lr=lr_fixed,
        max_epochs=max_epochs,
        Ee=Ee
    )

    train_acc = evaluate_model(X_train, y_train, w, b)
    test_acc = evaluate_model(X_test, y_test, w, b)
    full_acc = evaluate_model(X, y, w, b)

    results[name] = {
        "w": w,
        "b": b,
        "errors": errors,
        "epochs": epochs,
        "train_acc": train_acc,
        "test_acc": test_acc,
        "full_acc": full_acc
    }

print("РЕЗУЛЬТАТЫ ОБУЧЕНИЯ")
print("-" * 70)
print(f"{'Метод':<20} {'Эпохи':<10} {'Train acc':<12} {'Test acc':<12} {'Full acc':<12}")
print("-" * 70)

for name, res in results.items():
    print(f"{'name':<20} {res['epochs']:<10} {res['train_acc']:<12.4f} {res['test_acc']:<12.4f} {res['full_acc']:<12.4f}")

print("-" * 70)

# =====
# 11. График сходимости
# =====
plt.figure(figsize=(10, 6))

for name, res in results.items():
    plt.plot(range(1, len(res["errors"]) + 1), res["errors"], label=name)

plt.xlabel("Номер эпохи")
plt.ylabel("Суммарная ошибка Es")
plt.title("Сходимость обучения: MSE и BCE")
plt.legend()
plt.grid(True)

```

```
plt.tight_layout()
plt.show()
```

```
best_name = None
best_res = None
```

```
for name, res in results.items():
    if best_res is None:
        best_name = name
        best_res = res
    else:
        if res["test_acc"] > best_res["test_acc"]:
            best_name = name
            best_res = res
        elif res["test_acc"] == best_res["test_acc"] and res["epochs"] < best_res["epochs"]:
            best_name = name
            best_res = res
```

```
print(f"\nЛучшая конфигурация: {best_name}")
```

```
def true_or_function(x):
    return int(np.any(x == 1))
```

```
def user_mode(w, b):
    print(f"\nВведите {n} значений (0 или 1) через пробел:")
    raw = input().strip().split()

    if len(raw) != n:
        print(f"Ошибка: нужно ввести ровно {n} значений.")
        return

    try:
        x = np.array(list(map(int, raw))).reshape(1, -1)
    except ValueError:
        print("Ошибка: ввод должен содержать только числа 0 или 1.")
        return

    if not np.all((x == 0) | (x == 1)):
        print("Ошибка: допустимы только 0 и 1.")
        return

    prob = sigmoid(x @ w + b)[0, 0]
    pred_class = int(prob >= 0.5)
    true_class = true_or_function(x)
```

```
print(f"Вероятность  $\hat{y} = \{prob:.4f\}$ ")
print(f"Предсказанный класс = {pred_class}")
print("Совпадает с таблицей истинности" if pred_class == true_class else "Расхождение")
```

```
# Запуск режима пользователя
user_mode(best_res["w"], best_res["b"])
```

Результат:

РЕЗУЛЬТАТЫ ОБУЧЕНИЯ

Метод	Эпохи	Train acc	Test acc	Full acc
MSE fixed	200	0.9994	1.0000	0.9995
MSE adaptive	107	1.0000	1.0000	1.0000
BCE fixed	200	0.9994	1.0000	0.9995
BCE adaptive	139	1.0000	1.0000	1.0000

Лучшая конфигурация: MSE adaptive

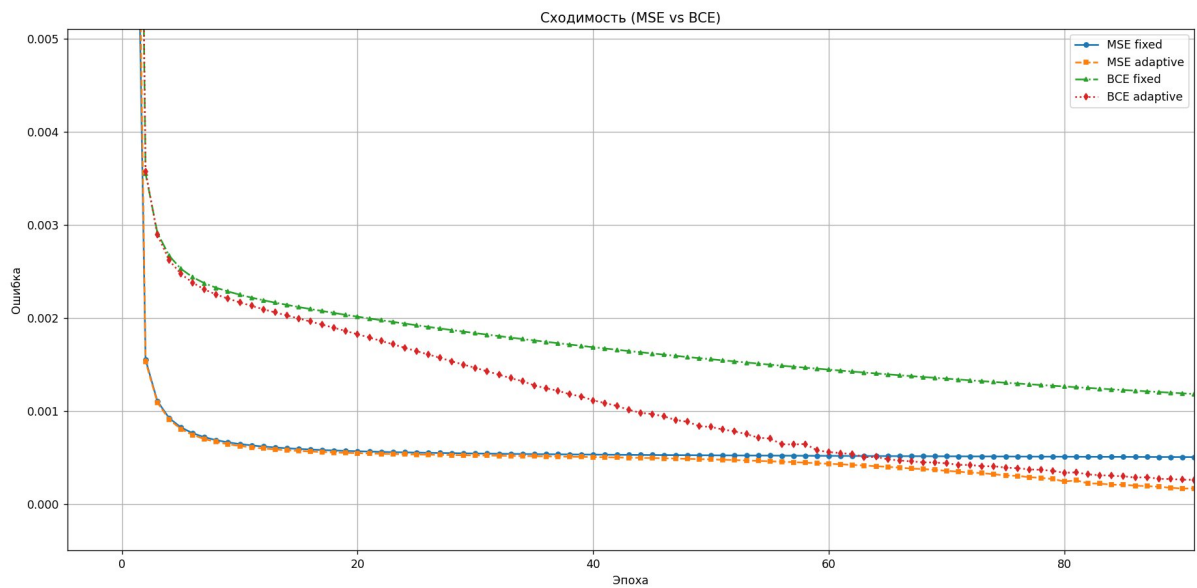
Введите 11 значений (0 или 1) через пробел:

0 1 1 1 0 0 1 1 1 1 0

Вероятность: 1.0000

Класс: 1

Совпадает с таблицей истинности



Вывод:

В ходе выполнения лабораторной работы был реализован однослойный персептрон с сигмоидной функцией активации и двумя функциями потерь: MSE и бинарной кросс-энтропией (BCE). Проведено сравнение их эффективности при решении задачи бинарной классификации логической функции AND от 9 переменных.

В результате экспериментов было установлено, что обучение с использованием функции потерь BCE сходится быстрее, чем с использованием MSE. Это связано с тем, что BCE сильнее штрафует уверенные, но неправильные предсказания, благодаря чему градиенты остаются значимыми даже при ошибках высокой уверенности, тогда как при MSE они могут быстро затухать.

Также было подтверждено, что бинарная кросс-энтропия лучше подходит для задач классификации, так как она непосредственно связана с вероятностной интерпретацией выходов модели и обеспечивает более корректную настройку весов при использовании сигмоидной функции активации.

При переходе от MSE к BCE наблюдалось улучшение обобщающей способности модели: увеличилась точность на тестовой выборке, а также процент правильно восстановленных значений полной таблицы истинности.

Использование адаптивного шага обучения положительно повлияло на процесс обучения. В большинстве случаев он позволял ускорить сходимость и избежать застревания в локальных колебаниях, по сравнению с фиксированным шагом.

Также было показано, что однослойного персептрона достаточно для реализации логической функции AND, так как она является линейно разделимой. Однако для более

сложных логических функций, например XOR, потребуется использование многослойных нейронных сетей.

Таким образом, можно сделать вывод, что применение функции потерь ВСЕ в сочетании с адаптивным шагом обучения является наиболее эффективным подходом для решения задач бинарной классификации.